Simulación de un ecosistema depredador–presa

Implementación secuencial y paralela en memoria compartida (OpenMP y Pthreads)

Carlos Orduz Samuel Rueda Daniela Téllez

Computación Paralela y Distribuida
5 de noviembre de 2026

Resumen

Este informe presenta la implementación de una simulación de un ecosistema con zorros, conejos y rocas sobre una matriz de $R \times C$ celdas. Se desarrolló una versión secuencial en C++17 y dos versiones paralelas para memoria compartida, una con OpenMP y otra con Pthreads, además de un visualizador web interactivo. El diseño paralelo se basa en un esquema de *decidir–recoger* (*gather*) en el que cada celda es escrita por un único hilo en cada paso. Por ello no se necesitan *locks* ni operaciones atómicas, y el resultado es determinista e idéntico al de la versión secuencial para cualquier número de hilos. Se verificó la corrección con el ejemplo del enunciado, con los 7 casos de prueba oficiales, con 232 pruebas automáticas y con una validación cruzada contra una segunda implementación independiente. Finalmente se analizan el *speedup* obtenido y la dinámica de las poblaciones.

Repositorio: `https://github.com/USUARIO/ecosystem-simulation`

Índice

1. Introducción **3**

2. Descripción del problema **3**
2.1. Reglas 3
2.2. Formato de entrada y salida 3

3. Interpretación de las reglas y verificación con el ejemplo **3**

4. Implementación secuencial **4**
4.1. Estructuras de datos 4
4.2. Algoritmo: decidir y recoger 4

5. Implementación paralela **5**
5.1. Por qué no hay condiciones de carrera 5
5.2. Versión OpenMP 5
5.3. Versión Pthreads 6

6. Visualizador **6**

7. Verificación **7**

8. Resultados experimentales **7**
8.1. Metodología 7
8.2. Análisis 9

9. Dinámica de poblaciones **10**
9.1. Sensibilidad a los parámetros 11

10. Conclusiones **12**

A. Compilación y uso **12**

B. Estructura del repositorio **13**

1. Introducción

El problema consiste en simular, durante `N_GEN` generaciones, un ecosistema habitado por dos especies cuya existencia es mutuamente dependiente. Los conejos se reproducen y sirven de alimento a los zorros. Los zorros necesitan comer conejos para no morir de hambre. Las rocas son obstáculos estáticos que modifican la dinámica local. El enunciado pide implementar primero una versión secuencial y después una versión paralela para memoria compartida, empleando procesos con memoria compartida, Pthreads u OpenMP.

Los entregables de este proyecto son:

- `ecosystem_seq`: versión secuencial.
- `ecosystem_omp`: versión paralela con OpenMP; la planificación se configura en tiempo de ejecución.
- `ecosystem_pthreads`: versión paralela con Pthreads; cada hilo es responsable de un bloque fijo de filas.
- Un visualizador web con grilla, estadísticas, parámetros ajustables y controles de tiempo.
- Scripts de prueba, de validación cruzada y de medición de rendimiento.

2. Descripción del problema

2.1. Reglas

Conejos. Se mueven en horizontal o en vertical a una celda adyacente vacía. Si no hay ninguna, permanecen en su lugar. Cuando han pasado `GEN_PROC_RABBITS` generaciones desde que nacieron o desde que procrearon por última vez y logran moverse, dejan una cría en su posición anterior, y ambos reinician su edad de procreación a cero.

Zorros. Intentan comer un conejo adyacente. Si no hay ninguno, mueren cuando han pasado `GEN_FOOD_FOXES` generaciones sin comer; en caso contrario, intentan moverse a una celda vacía. Procrean de forma análoga a los conejos, con el parámetro `GEN_PROC_FOXES`.

Rocas. No se mueven y ningún animal puede ocupar su celda.

Selección de celda. Las P celdas candidatas se numeran desde 0 en sentido horario (N, E, S, O), y se elige la celda $(G + X + Y)$ mód P, donde G es la generación actual y (X, Y) la posición del animal.

Conflictos. En cada generación se mueven primero todos los conejos y luego todos los zorros, como si dentro de cada grupo todos se movieran al mismo tiempo. Si varios animales llegan a la misma celda, sobrevive el de mayor edad de procreación; entre zorros empatados, el menos hambriento.

2.2. Formato de entrada y salida

La primera línea contiene `GEN_PROC_RABBITS GEN_PROC_FOXES GEN_FOOD_FOXES N_GEN R C N`, y le siguen N líneas `OBJETO X Y`, con `OBJETO` $\in$ {`ROCK`, `FOX`, `RABBIT`}. La salida tiene el mismo formato, con `N_GEN`$= 0$, y describe el estado final. La entrada se lee de *stdin* y la salida se escribe en *stdout*.

3. Interpretación de las reglas y verificación con el ejemplo

El enunciado deja algunos detalles abiertos: en qué momento se incrementan los contadores y si las comparaciones son estrictas. Se fijaron analizando el ejemplo generación por generación. La única interpretación consistente con las seis generaciones mostradas es la siguiente:

1. X es la fila e Y la columna; el origen $(0, 0)$ es la esquina superior izquierda.
2. Al comenzar su sub-fase, cada animal incrementa su edad de procreación en 1. Cada zorro incrementa también su contador de hambre.
3. Un animal procrea si **se mueve** y su edad, ya incrementada, es **estrictamente mayor** que `GEN_PROC_*`. Por ejemplo, con `GEN_PROC_RABBITS`= 2, un conejo nacido en la generación 0 procrea por primera vez al moverse en la generación 2.
4. Un zorro sin conejos adyacentes muere si su hambre, ya incrementada, es $\geq$ `GEN_FOOD_FOXES`. Con `GEN_FOOD_FOXES`= 3, un zorro que nunca come muere en la generación 2, como el zorro en $(1, 2)$ del ejemplo. Al comer, su hambre vuelve a 0.
5. En un conflicto se comparan los valores con los que cada animal llegaría a la celda, es decir, después de aplicar el reinicio por procreación o por comida.

Con estas reglas, la versión secuencial con la opción `--print-grid` reproduce exactamente la secuencia del enunciado (figura 1) y su salida final.

```
  Gen 0       Gen 1       Gen 2       Gen 3       Gen 4       Gen 5       Gen 6
|* R F|     |*R F |     |* F  |     |*  F |     |*   F|     |*    |     |*    |
|F   F|     |   F |     |  F  |     |     |     |     |     |     |     |     |
|    *|     |F   *|     |    *|     |    *|     |    *|     |    *|     |    *|
|R    |     | R  F|     |F    |     |     |     | R   |     |R    |     |R    |
|R   F|     | R   |     |  R F|     |FRR  |     | F R |     | FF  |     |F  F |
```

Figura 1: Evolución del ejemplo del enunciado producida por `ecosystem_seq -g`. Coincide celda a celda con la del enunciado. Salida final: `2 4 3 0 5 5 5`, `ROCK 0 0`, `ROCK 2 4`, `RABBIT 3 0`, `FOX 4 0`, `FOX 4 3`.

4. Implementación secuencial

4.1. Estructuras de datos

El mundo se almacena como una *estructura de arreglos* (SoA) en orden de filas: un arreglo `type` de 1 byte por celda (vacío, roca, conejo o zorro) y dos arreglos `age` y `hunger` de 16 bits. Se usan dos buffers, A y B. La sub-fase de conejos lee A y escribe B, y la de zorros lee B y escribe A. Así, al terminar cada generación el estado vuelve a estar en A y no hace falta intercambiar punteros. Un arreglo auxiliar `mv` de 1 byte por celda guarda la decisión de cada animal: dirección 0–3, quedarse o morir. En total se usan 11 bytes por celda.

Los contadores de 16 bits se saturan en 65 535. Esto solo podría afectar a un conflicto entre dos animales que llevaran más de 65 535 generaciones sin procrear. Se eligió este tamaño porque el kernel está limitado por memoria (sección 8).

4.2. Algoritmo: decidir y recoger

El enunciado exige que el orden en que se procesan los animales no afecte al resultado. Para garantizarlo, cada sub-fase se divide en dos pasos:

1. **Decidir.** Cada animal examina la *instantánea* del mundo al inicio de la sub-fase y guarda su decisión en `mv`. Solo escribe en su propia celda.
2. **Recoger** (*gather*). Cada celda del buffer destino calcula su contenido: mira si su propio animal se queda, se va dejando una cría, se va o muere, y *recoge* a los animales vecinos cuya decisión apunta hacia ella, aplicando las reglas de conflicto. Las nuevas edades y hambres se derivan de la instantánea; por ejemplo, un zorro que llega a una celda que tenía un conejo comió.

Algorithm 1 Una generación G (cada bucle recorre un rango de filas)

for all celdas i con conejo en A **do** ▷ decidir conejos
  $E \leftarrow$ vecinos vacíos de i en A (orden N, E, S, O)
  $mv[i] \leftarrow |E| > 0 \ ? \ E[(G + x + y) \bmod |E|] : \text{QUEDARSE}$
end for
for all celdas i **do** ▷ recoger conejos: $A \rightarrow B$
  copiar rocas y zorros; resolver el conejo propio (se queda / deja cría / se va)
  si i está vacía: entre los conejos vecinos j con $mv[j]$ hacia i, gana el de mayor edad
end for
for all celdas i con zorro en B **do** ▷ decidir zorros
  $Rb \leftarrow$ vecinos con conejo; $E \leftarrow$ vecinos vacíos
  if $|Rb| > 0$ **then**
    $mv[i] \leftarrow Rb[(G + x + y) \bmod |Rb|]$
  else if $hambre(i) + 1 \geq$ `GEN_FOOD_FOXES` **then**
    $mv[i] \leftarrow \text{MORIR}$
  else
    $mv[i] \leftarrow |E| > 0 \ ? \ E[(G + x + y) \bmod |E|] : \text{QUEDARSE}$
  end if
end for
for all celdas i **do** ▷ recoger zorros: $B \rightarrow A$
  resolver el zorro propio; si i está vacía o tiene conejo, recoger a los zorros que llegan
  gana la mayor edad; en caso de empate, el menor hambre
end for

Cada generación cuesta $\Theta(R \cdot C)$ operaciones, con un número constante de accesos a vecinos por celda.

5. Implementación paralela

5.1. Por qué no hay condiciones de carrera

En el esquema *decidir–recoger*, en cada paso cada celda de salida es escrita por **un único hilo**, el que tiene asignada esa fila, y solo se leen datos que nadie modifica durante ese paso: la instantánea y `mv`. Los movimientos que cruzan la frontera entre dos hilos no requieren intercambio explícito de mensajes ni regiones *halo*: basta con una **barrera** entre pasos para que las decisiones del hilo vecino sean visibles. Este diseño tiene varias ventajas:

- no hay *locks*, operaciones atómicas ni secciones críticas;
- el resultado es **determinista** e idéntico al secuencial con cualquier número de hilos y cualquier planificación;
- el mismo código de los kernels sirve para las tres versiones, que solo difieren en cómo reparten las filas.

La alternativa ingenua, que cada animal *empuje* su movimiento a la celda destino, requeriría sincronizar la escritura en las celdas de las fronteras entre hilos, mediante *locks* por celda o fases adicionales de intercambio con los vecinos.

5.2. Versión OpenMP

Se crea una única región paralela para toda la simulación, de modo que los hilos se crean una sola vez. Cada paso es un `omp for` sobre las filas, y su barrera implícita sincroniza los pasos:

```
#pragma omp parallel
for (int G = 0; G < NG; ++G) {
```

```
    #pragma omp for schedule(runtime)
    for (int r = 0; r < R; ++r) sim.rabbits_decide(G, r, r + 1);
    #pragma omp for schedule(runtime)
    for (int r = 0; r < R; ++r) sim.rabbits_resolve(r, r + 1);
    #pragma omp for schedule(runtime)
    for (int r = 0; r < R; ++r) sim.foxes_decide(G, r, r + 1);
    #pragma omp for schedule(runtime)
    for (int r = 0; r < R; ++r) sim.foxes_resolve(r, r + 1);
}
```

Con `schedule(runtime)`, la planificación se elige mediante `OMP_SCHEDULE` sin recompilar. `static` asigna a cada hilo un bloque fijo de filas, mientras que `dynamic,k` y `guided` reparten las filas de forma **adaptativa**: un hilo que termina antes toma más trabajo, lo que balancea la carga cuando la densidad de animales no es uniforme. Si `OMP_SCHEDULE` no está definida, el programa usa `static`, porque el valor por defecto de libgomp (`dynamic,1`) añade una operación atómica por fila. Las estadísticas por generación se calculan con una reducción (`reduction(+:rabbits,foxes)`).

5.3. Versión Pthreads

Es la estrategia más simple sugerida por el enunciado: cada *worker* es responsable de una porción fija del mundo, un bloque contiguo de $\lceil R/T \rceil$ filas. Los hilos se sincronizan con una `pthread_barrier_t` después de cada paso, es decir, cuatro barreras por generación. Así, al final de cada paso, los datos de los animales que cruzaron de la porción de un *worker* a la de otro quedan sincronizados.

6. Visualizador

El visualizador (`visualizer/index.html`) es una aplicación web autónoma que cumple los requisitos de implementación del enunciado (figura 2):

- **Visualización de la grilla** con todas las entidades. Al pasar el ratón sobre una celda se muestran su edad de procreación y su hambre.
- **Estadísticas**: contadores de cada especie, gráfica de población por generación con cursor, tabla de las últimas generaciones y exportación a CSV.
- **Parámetros ajustables**: tamaño, porcentajes iniciales de rocas, conejos y zorros, semilla, y las tres tasas de reproducción y hambre, que pueden aplicarse *en caliente* sin reiniciar.
- **Controles de tiempo**: reproducir/pausar, paso a paso, más lento/más rápido (de 1 generación por segundo hasta el máximo) y reiniciar, con atajos de teclado.
- Carga de archivos en el formato del enunciado y descarga del estado actual en ese mismo formato.

Su motor (`ecosystem.js`) implementa las reglas de forma *independiente*, con un estilo "push" en lugar de "gather". Por eso se usa también para validar de forma cruzada la versión C++ (sección 7).



Figura 2: Visualizador web: parámetros (izquierda), mundo (centro) y estadísticas de población (derecha).

7. Verificación

1. **Ejemplo del enunciado.** Las tres versiones producen la salida esperada, y la evolución coincide generación por generación (figura 1).
2. **Casos de prueba oficiales.** Las tres versiones, con 1, 2, 3, 4 y 8 hilos, reproducen exactamente la salida esperada de los 7 casos oficiales del curso: 5×5, 10×10, 20×20, 100×100, dos mundos 100×100 desbalanceados y 200×200.
3. **Paralelo frente a secuencial.** En seis mundos aleatorios, algunos degenerados (1×300 y 300×1), se exige que OpenMP, con 1, 2, 3, 5, 8 y 16 hilos y las planificaciones `static`, `dynamic,1` y `guided,4`, y Pthreads produzcan *exactamente* la misma salida que la versión secuencial. Con el punto anterior, `scripts/run_tests.sh` supera 232 de 232 comprobaciones.
4. **Validación cruzada** (`scripts/crosscheck.js`). En 200 mundos aleatorios con parámetros variados, la versión C++ y el motor JavaScript independiente producen el mismo estado final. Como ambas implementaciones fueron escritas con estrategias distintas, esto da confianza en que las reglas se interpretaron igual en las dos.

8. Resultados experimentales

8.1. Metodología

- Equipo: AMD Ryzen 9 8940HX (16 núcleos / 32 hilos), 32 GB RAM.
- Software: Windows 11; GCC 16.1 (MinGW-w64) con OpenMP 5.2; opciones `-O3` y `-march=native`.
- Mundos generados con `gen_input` (semilla 7): 5 % rocas, 30 % conejos, 5 % zorros; `GEN_PROC_RABBITS`=5, `GEN_PROC_FOXES`=12, `GEN_FOOD_FOXES`=12.
- Tamaños: 500×500 (2000 generaciones), 1000×1000 (500 generaciones) y 2000×2000 (200 generaciones). Los dos primeros casos suman el mismo trabajo total, $5 \cdot 10^8$ actualizaciones de celda, y el tercero $8 \cdot 10^8$.

- Se mide solo la simulación, sin la lectura ni la escritura de archivos. Cada punto es el **mínimo de 3 ejecuciones**. Speedup $S_p = T_{\text{sec}}/T_p$; eficiencia $E_p = S_p/p$.

Tabla 1: Tiempos (s) para el mundo 500×500 (2000 generaciones). Versión secuencial: 6.377 s.

Hilos	OMP static	OMP dynamic,16	OMP guided	Pthreads
1	6.519	6.700	6.390	6.304
2	3.987	4.300	4.035	4.150
4	2.561	2.615	2.448	2.419
8	2.059	1.893	1.814	2.224
12	2.191	2.096	2.102	2.531
16	2.775	2.138	2.037	2.720
24	2.428	2.480	2.523	3.487
32	3.008	3.190	3.579	4.752

Tabla 2: Tiempos (s) para el mundo 1000×1000 (500 generaciones). Versión secuencial: 6.848 s.

Hilos	OMP static	OMP dynamic,16	OMP guided	Pthreads
1	6.305	6.420	6.270	6.307
2	3.721	3.752	3.646	3.887
4	2.298	2.095	2.012	2.097
8	1.469	1.446	1.868	1.540
12	1.592	2.879	2.207	3.071
16	1.770	1.729	1.951	1.831
24	1.690	1.779	2.074	3.458
32	2.148	3.321	1.874	2.252

Tabla 3: Tiempos (s) para el mundo 2000×2000 (200 generaciones). Versión secuencial: 10.311 s.

Hilos	OMP static	OMP dynamic,16	OMP guided	Pthreads
1	10.335	9.939	10.406	10.618
2	5.418	5.649	6.591	6.526
4	3.382	3.507	3.376	3.962
8	2.437	2.498	3.147	2.825
12	3.096	3.302	2.874	2.878
16	2.721	2.878	2.638	2.844
24	2.722	2.721	2.661	3.019
32	2.781	2.818	2.783	3.124

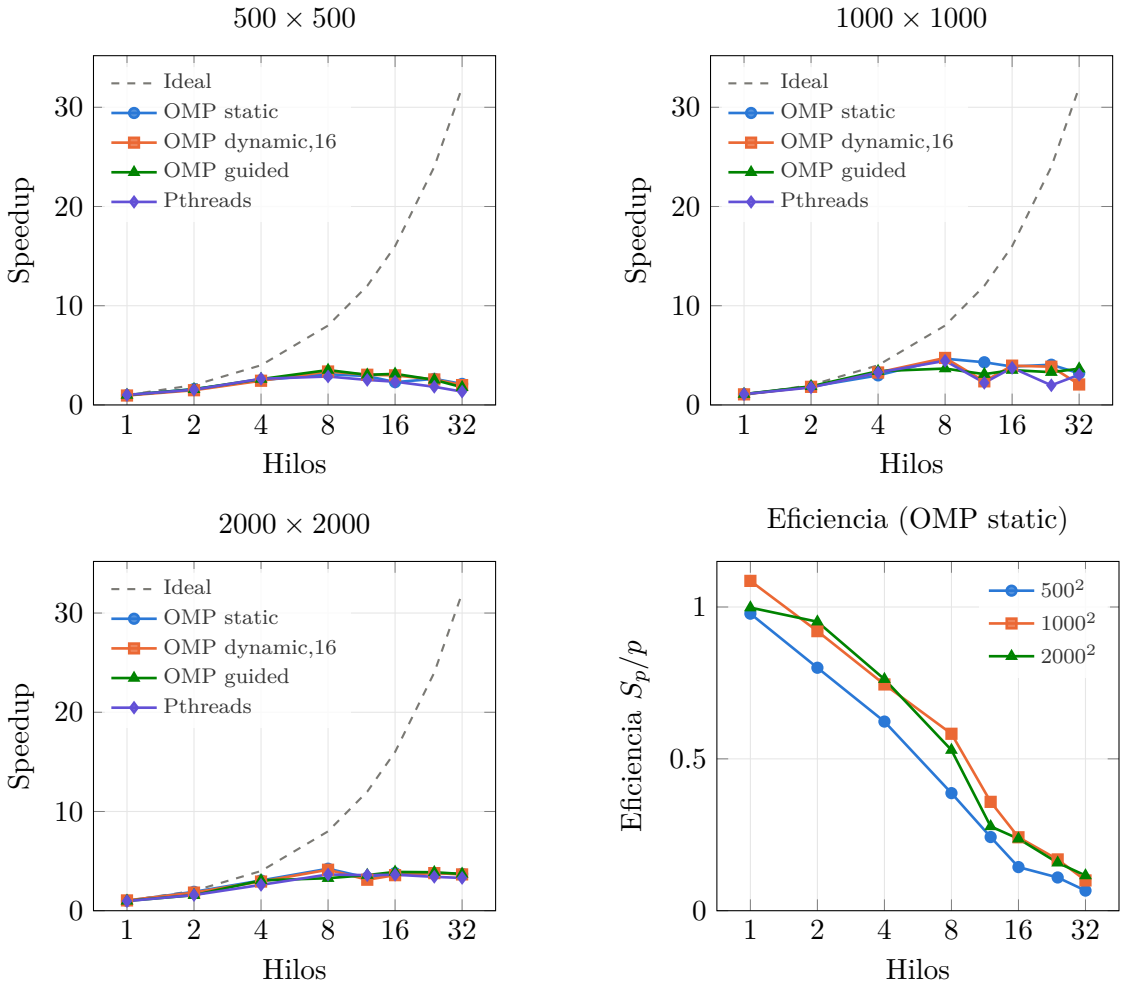


Figura 3: Speedup respecto de la versión secuencial y eficiencia de OpenMP `static`.

8.2. Análisis

Sobrecosto de la paralelización. Con un hilo, las versiones OpenMP y Pthreads tardan prácticamente lo mismo que la secuencial: las diferencias son menores al 5 % y están dentro del ruido de medición. Esto confirma que crear una única región paralela y llamar a los kernels por filas no añade un costo apreciable.

Escalabilidad. Para pocos hilos la aceleración es buena: con 2 hilos se obtiene un speedup de 1.6–1.9 y con 4 hilos de 2.6–3.4. El mejor resultado se alcanza alrededor de **8 hilos**, con un speedup de aproximadamente **4.7** en el mundo de 1000 × 1000, 4.2 en el de 2000 × 2000 y 3.5 en el de 500 × 500. A partir de 8–12 hilos el tiempo deja de mejorar, y con 24–32 hilos incluso empeora. Varias causas contribuyen a este comportamiento:

- **Kernel limitado por memoria.** Cada paso hace muy poco cómputo por byte transferido: compara tipos de celdas vecinas y copia el estado de un buffer a otro. Un microbenchmark de copia de arreglos en esta misma máquina deja de escalar a partir de 8 hilos, igual que la simulación.
- **Hilos lógicos (SMT).** Con más de 16 hilos, dos hilos comparten un mismo núcleo físico y sus recursos de caché y de carga/almacenamiento, lo que no aporta beneficio en un código limitado por memoria.
- **Barreras.** Cada generación tiene 4 barreras. En el mundo de 500 × 500 una generación dura apenas unos milisegundos, así que el costo de sincronizar 16–32 hilos pesa proporcio-

nalmente más. Por eso es el caso que peor escala.

- **Plataforma.** Las mediciones se hicieron en un portátil con Windows y el plan de energía "Equilibrado". La frecuencia del procesador baja cuando se activan muchos núcleos, y el planificador de Windows migra hilos entre los dos complejos de núcleos (CCD) del Ryzen. Incluso un bucle puramente de cómputo, sin acceso a memoria, alcanzó solo $\approx 6\times$ con 16 hilos en este equipo. Esto también explica la variabilidad observada; por ejemplo, algunas configuraciones con 12 o 24 hilos son notablemente más lentas que sus vecinas.

Planificación. Las tres planificaciones de OpenMP rinden de forma muy parecida, sin una ganadora consistente. Es lo esperado: en el esquema *decidir–recoger* cada fila tiene un costo casi constante, porque se recorren todas las celdas sin importar cuántos animales haya, de modo que el reparto `static` ya está balanceado. Las planificaciones adaptativas (`dynamic`, `guided`) solo compensarían si la densidad de animales hiciera muy desigual el trabajo por fila. Por otro lado, tienen la desventaja de que un hilo puede recibir filas distintas en cada paso, lo que empeora la localidad de caché. Pthreads, con bloques estáticos de filas, se comporta como OpenMP `static` hasta 8 hilos. Con más hilos sufre algo más, porque su barrera (`pthread_barrier` de winpthreads) es menos eficiente que la de libgomp.

Tamaño del problema. El mundo de 1000×1000 es el que mejor escala: su estado de unos 11 MB cabe en la caché L3. El de 2000×2000 (unos 44 MB) excede la caché y depende más del ancho de banda de la memoria principal, y en el de 500×500 domina el costo de las barreras.

9. Dinámica de poblaciones

La figura 4 muestra la evolución de ambas especies en un mundo de 200×200, en la que se distinguen tres comportamientos:

- **Ciclo depredador–presa inicial.** Al comienzo abundan los conejos, así que los zorros comen y se reproducen: su población alcanza un máximo hacia la generación 16 y hace caer la de conejos de 12 000 a menos de 4 000 hacia la generación 40. Con menos presas, muchos zorros mueren de hambre y los conejos se recuperan. El máximo de zorros ocurre *después* del de conejos, el desfase característico de las ecuaciones de Lotka–Volterra.
- **Oscilación rápida.** Durante toda la simulación ambas poblaciones oscilan con un período de unas 6 generaciones. Los conejos nacidos en la misma generación procrean juntos cada `GEN_PROC_RABBITS`+1 generaciones, lo que produce pulsos de nacimientos a los que los zorros responden.
- **Cuasi-equilibrio.** Pasado el transitorio, la media global se estabiliza. Los ciclos siguen ocurriendo de forma *local* en distintas regiones del mundo, pero en momentos distintos, de modo que se compensan en el total.

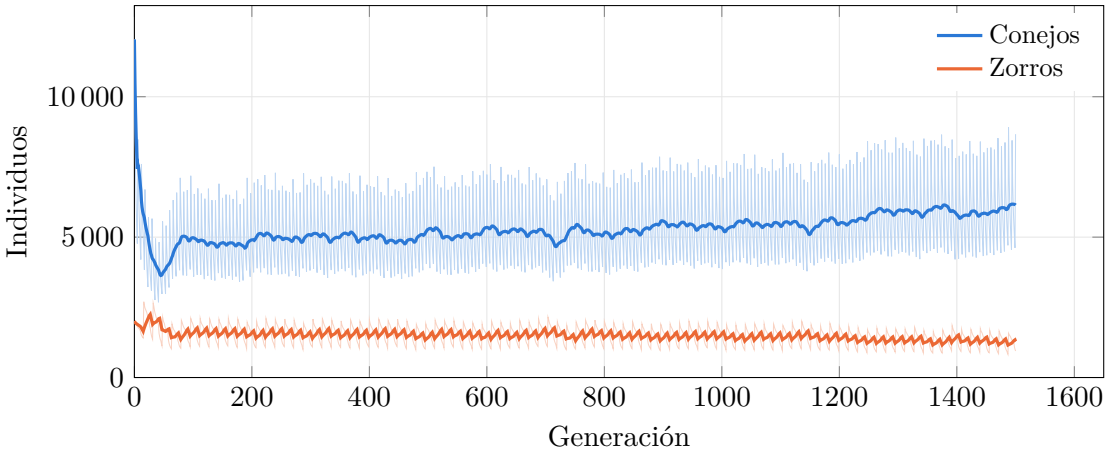


Figura 4: Población por generación (200 × 200; 5 % rocas, 30 % conejos, 5 % zorros; `GEN_PROC_RABBITS`=5, `GEN_PROC_FOXES`=15, `GEN_FOOD_FOXES`=10). En tono claro, el valor de cada generación; en trazo grueso, la media móvil de 12 generaciones, que filtra la oscilación rápida de período ≈ 6 causada por los nacimientos sincronizados de los conejos.

9.1. Sensibilidad a los parámetros

Para estudiar cómo afectan los parámetros a la estabilidad del ecosistema, se repitió la simulación variando un solo parámetro a la vez (figura 5).

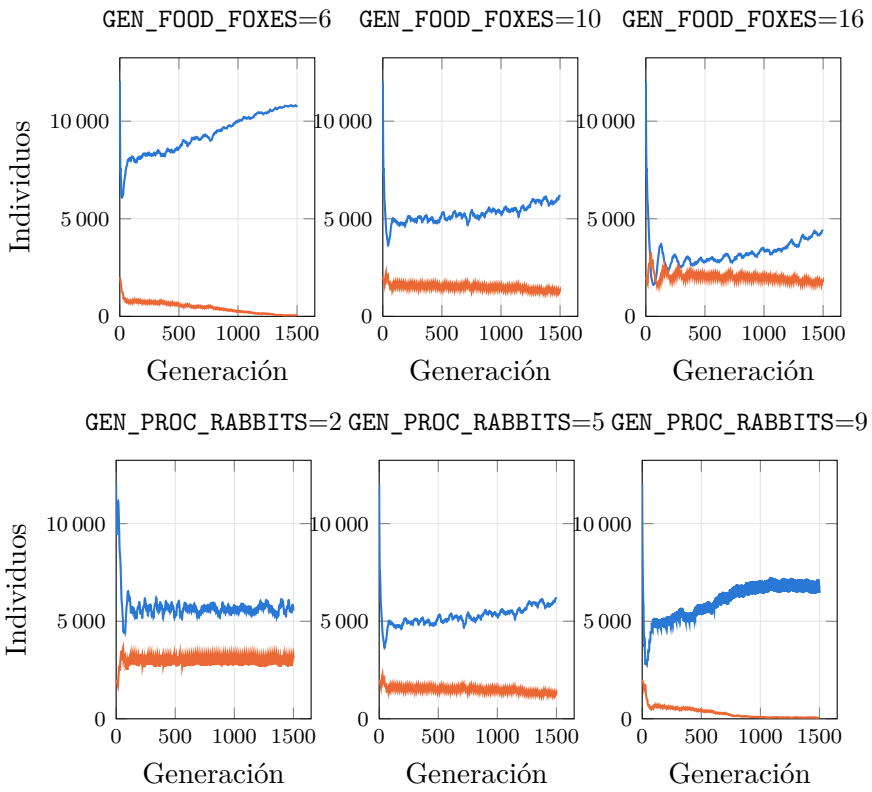


Figura 5: Sensibilidad de la dinámica a `GEN_FOOD_FOXES` (arriba) y a `GEN_PROC_RABBITS` (abajo), con media móvil de 12 generaciones. El resto de parámetros es igual al de la figura 4. Conejos en azul, zorros en naranja.

Los resultados muestran que los parámetros determinan si el ecosistema es estable:

- **`GEN_FOOD_FOXES` bajo (6).** Los zorros mueren de hambre antes de encontrar presas suficientes. Su población cae de forma sostenida hasta casi extinguirse (43 individuos en la generación 1500), y los conejos, con cada vez menos depredadores, crecen hasta superar los 10 000.
- **`GEN_FOOD_FOXES` alto (16).** Los zorros resisten largos periodos sin comer. El ciclo depredador–presa es el más visible: en las primeras 300 generaciones se observan varias oscilaciones amortiguadas, con los máximos de zorros desfasados respecto de los de conejos. Después, ambas especies se estabilizan, con una población de zorros alta (unos 2 000) y los conejos en un nivel más bajo que con los parámetros de referencia.
- **`GEN_PROC_RABBITS` bajo (2).** Los conejos se reproducen muy rápido y sostienen la mayor población de zorros de todos los experimentos (unos 3 000), con fluctuaciones rápidas y amplias en ambas especies.
- **`GEN_PROC_RABBITS` alto (9).** Los conejos se reproducen lentamente. Los zorros no encuentran alimento suficiente y quedan al borde de la extinción (43 individuos en la generación 1500), mientras los conejos, liberados de la depredación, aumentan.

En resumen, el ecosistema es estable solo cuando la tasa de reproducción de las presas y la resistencia al hambre de los depredadores están equilibradas. Si no lo están, los zorros se extinguen. Las rocas actúan como refugios locales: fragmentan el espacio y dificultan que los zorros alcancen a los conejos, lo que ayuda a que las presas sobrevivan a los periodos de mayor depredación. Estos experimentos pueden reproducirse de forma interactiva en el visualizador modificando los parámetros durante la ejecución.

10. Conclusiones

- Se implementó la simulación completa con las reglas del enunciado. Las tres versiones reproducen exactamente el ejemplo y los 7 casos oficiales, y coinciden entre sí y con una implementación independiente.
- El esquema *decidir–recoger* hace que la paralelización sea segura por construcción: cada celda tiene un único escritor por paso, por lo que no se necesitan *locks* ni operaciones atómicas, y basta con barreras para sincronizar los movimientos entre las porciones de cada hilo. El resultado es determinista e independiente del número de hilos.
- La versión paralela reduce el tiempo de forma significativa, aunque su escalabilidad está limitada por el ancho de banda de memoria y por el costo de sincronización: el kernel realiza muy pocas operaciones por byte. La planificación `static` es la más adecuada, porque el trabajo por fila es casi uniforme. Las planificaciones adaptativas añaden *overhead* sin una ganancia de balanceo apreciable en estos mundos.
- El visualizador permite observar los ciclos depredador–presa y cómo los parámetros afectan a la estabilidad del ecosistema.

Trabajo futuro. Fusionar los pasos de *decidir* y *recoger* recalculando las decisiones de los vecinos, lo que reduciría las barreras de cuatro a dos por generación; usar bloques 2D (*tiling*) para mejorar la localidad de caché; e implementar una versión de memoria distribuida con MPI, con intercambio de filas fantasma (*halo*).

A. Compilación y uso

```
make                        # o: bash build.sh  /  .\build.ps1  (Windows)
./bin/ecosystem_seq          < tests/inputs/ex_enunciado.in
./bin/ecosystem_omp -t 8     < entrada.in > salida.out
```

```
./bin/ecosystem_pthreads -t 8 --time < entrada.in > salida.out
OMP_SCHEDULE="guided" ./bin/ecosystem_omp -t 16 -s poblacion.csv < entrada.in
./bin/gen_input 1000 1000 500 5 30 5 5 12 12 7 > mundo.in   # mundo aleatorio
bash scripts/run_tests.sh       # pruebas de correccion
node scripts/crosscheck.js      # validacion cruzada C++ vs JavaScript
bash scripts/benchmark.sh 3     # tiempos -> report/data/*.csv
bash scripts/population.sh      # dinamica de poblaciones -> report/data/pop_*.csv
```

B. Estructura del repositorio

```
src/          ecosystem.hpp/.cpp (nucleo), cli.hpp, main_seq/omp/pthreads.cpp
tools/        gen_input.cpp (generador de mundos)
tests/        inputs/*.in, expected/*.out, allgen/ (casos oficiales)
ecosistemas/  casos de prueba oficiales del curso (copia original)
scripts/      run_tests.sh, crosscheck.js, benchmark.sh, population.sh
visualizer/   index.html, style.css, app.js, ecosystem.js
report/       main.tex, analisis.tex, analisis_poblacion.tex, data/*.csv, figures/
```